

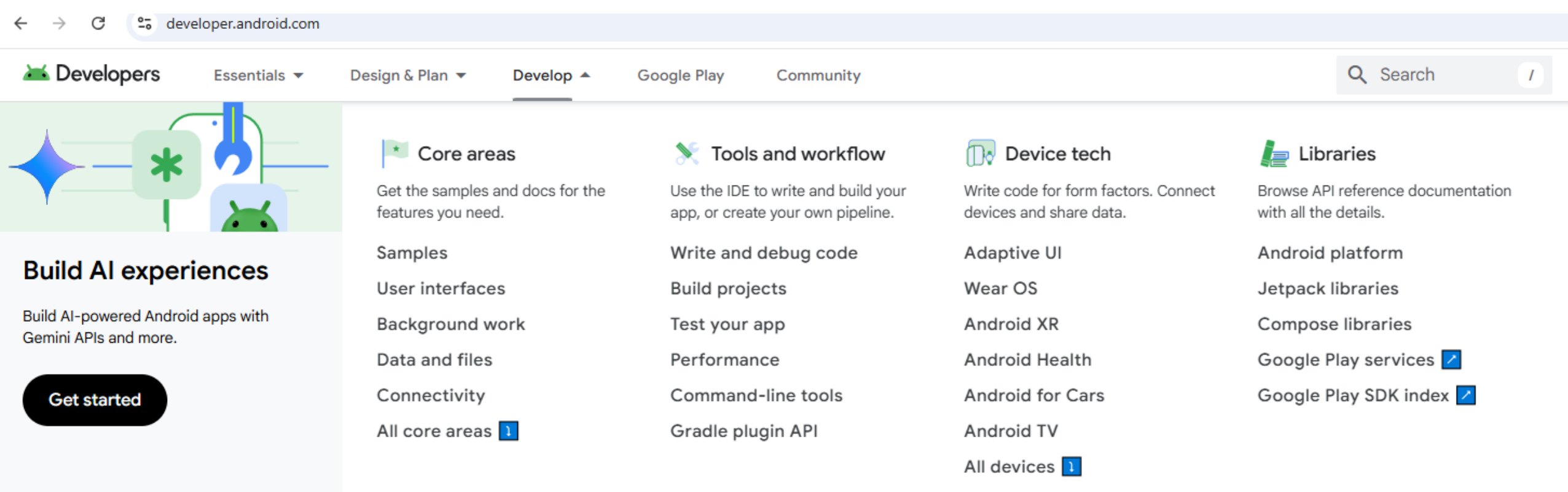
Android

1.

Bolla,
Kálmán

Compulsory readings

- Official Android webpage:
 - <http://developer.android.com/index.html>



The screenshot shows the top portion of the Android Developer website. The browser's address bar displays 'developer.android.com'. The navigation bar includes links for 'Developers', 'Essentials', 'Design & Plan', 'Develop' (which is the active tab), 'Google Play', and 'Community'. A search bar is located on the right side of the navigation bar. Below the navigation bar, there are several sections. On the left, there is a 'Build AI experiences' section with a 'Get started' button. To the right of this, there are five main columns of content: 'Core areas', 'Tools and workflow', 'Device tech', and 'Libraries'. Each column has a list of links to various resources. The 'Core areas' column includes links to 'Samples', 'User interfaces', 'Background work', 'Data and files', 'Connectivity', and 'All core areas'. The 'Tools and workflow' column includes links to 'Write and debug code', 'Build projects', 'Test your app', 'Performance', 'Command-line tools', and 'Gradle plugin API'. The 'Device tech' column includes links to 'Adaptive UI', 'Wear OS', 'Android XR', 'Android Health', 'Android for Cars', 'Android TV', and 'All devices'. The 'Libraries' column includes links to 'Browse API reference documentation', 'Android platform', 'Jetpack libraries', 'Compose libraries', 'Google Play services', and 'Google Play SDK index'.

← → ↻ 🔍 developer.android.com

Developers Essentials ▾ Design & Plan ▾ **Develop** ▲ Google Play Community 🔍 Search /

Build AI experiences
Build AI-powered Android apps with Gemini APIs and more.
Get started

Core areas
Get the samples and docs for the features you need.
Samples
User interfaces
Background work
Data and files
Connectivity
All core areas 1

Tools and workflow
Use the IDE to write and build your app, or create your own pipeline.
Write and debug code
Build projects
Test your app
Performance
Command-line tools
Gradle plugin API

Device tech
Write code for form factors. Connect devices and share data.
Adaptive UI
Wear OS
Android XR
Android Health
Android for Cars
Android TV
All devices 1

Libraries
Browse API reference documentation with all the details.
Android platform
Jetpack libraries
Compose libraries
Google Play services 🔗
Google Play SDK index 🔗

Official Android webpage



- Sample applications:
 - <https://developer.android.com/samples>
- Android guides:
 - <https://developer.android.com/guide>
- Android documentation:
 - <https://developer.android.com/reference>

Samples

A selection of code samples and templates for you to use to accelerate your app development. Browse samples to learn how to build different components for your applications.

Filter By

Language

Select all

☐ C++
☐ Java
☐ Kotlin

Topic

Select all

☐ Architecture
☐ Building
☐ Camera & Media

 Search by keyword

ACTION_OPEN_DOCUMENT

This sample demonstrates how to display PDF document on screen using the PdfRenderer introduced in Android 5.0 Lollipop.

ContentResolver
DocumentsContract
PdfRenderer

Android StorageStorageKotlin

 Intermediate

ACTION_OPEN_DOCUMENT_TREE

A proof of concept file manager utilizing the ACTION_OPEN_DOCUMENT_TREE intent introduced with Android...

ContentResolver
DocumentsContract

Android StorageStorageKotlin

 Intermediate

ActivitySceneTransitionnBasic

This sample shows how to use scene transitions from one Activity to another in Lollipop. Uses a combination of...

Transition
ActivityOptions

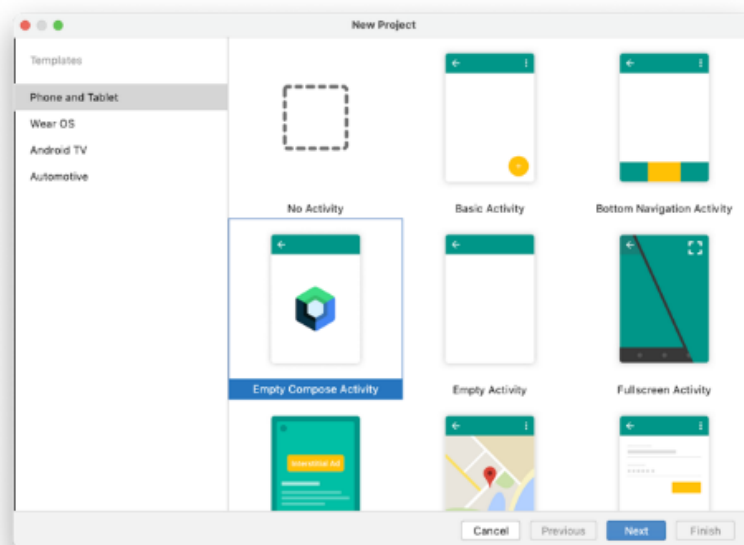
Android UI ViewsJava

 Intermediate



Get started with Android

To get started, do a quick codelab or dive deeper into a training course for an overview of key Android development topics.



Create a Hello World app

Follow this introductory codelab with step-by-step instructions to make a simple Hello World app.

[Start codelab](#)

Filter

Android API Reference

Overview

Android Platform

Jetpack

AndroidX

Jetpack Compose

Constraint Layout

Constraint Layout in Compose

Databinding Library

Device Libraries

Android Automotive Library

Other Libraries

Android Gradle Plugin

Material Components

Android Developers > Develop > API reference

Was this helpful?  

Android API reference

Start building your Android app with the Android Platform APIs. They are available in [Kotlin](#) and [Java](#).

★ **Note:** Many Kotlin reference topics are derived from Java-based source code. This means that some Kotlin reference topics might contain Java code snippets.

These additional libraries make it easy to add additional functionality and features to your app.

Jetpack

AndroidX

Refactored versions of the Android APIs that are not bundled with the operating system.
[Reference](#) [User Guide](#)

Android introduction

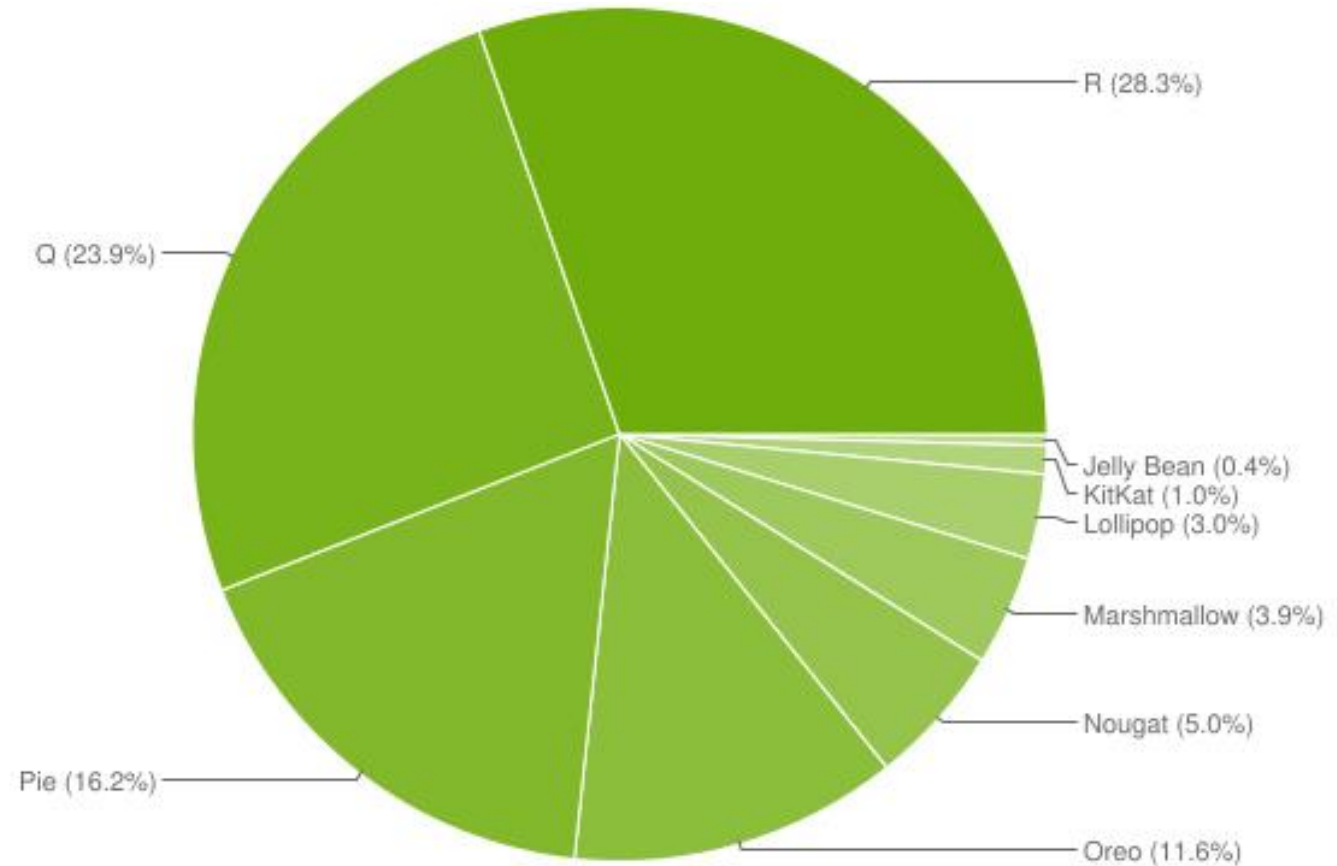
Android introduction

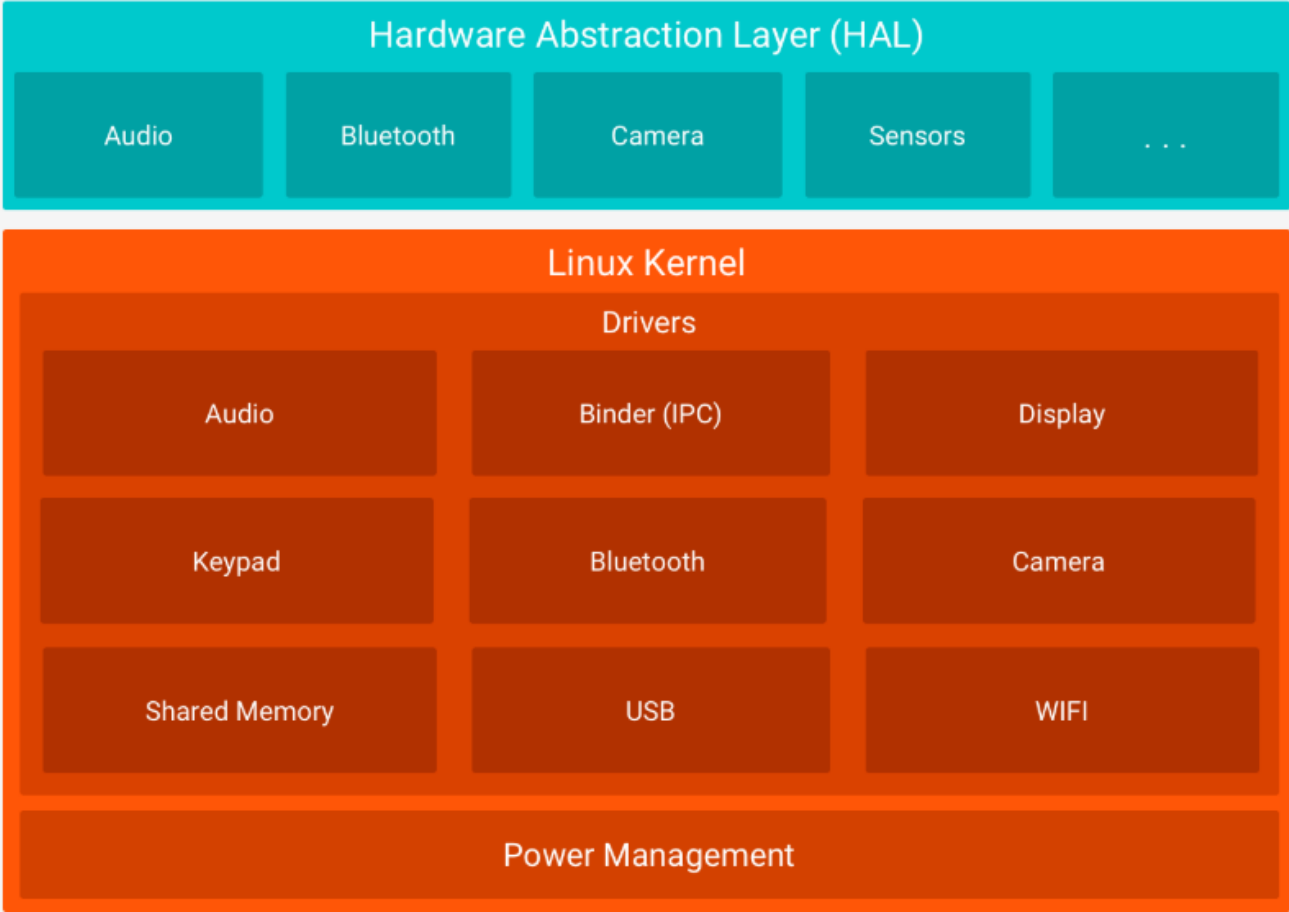
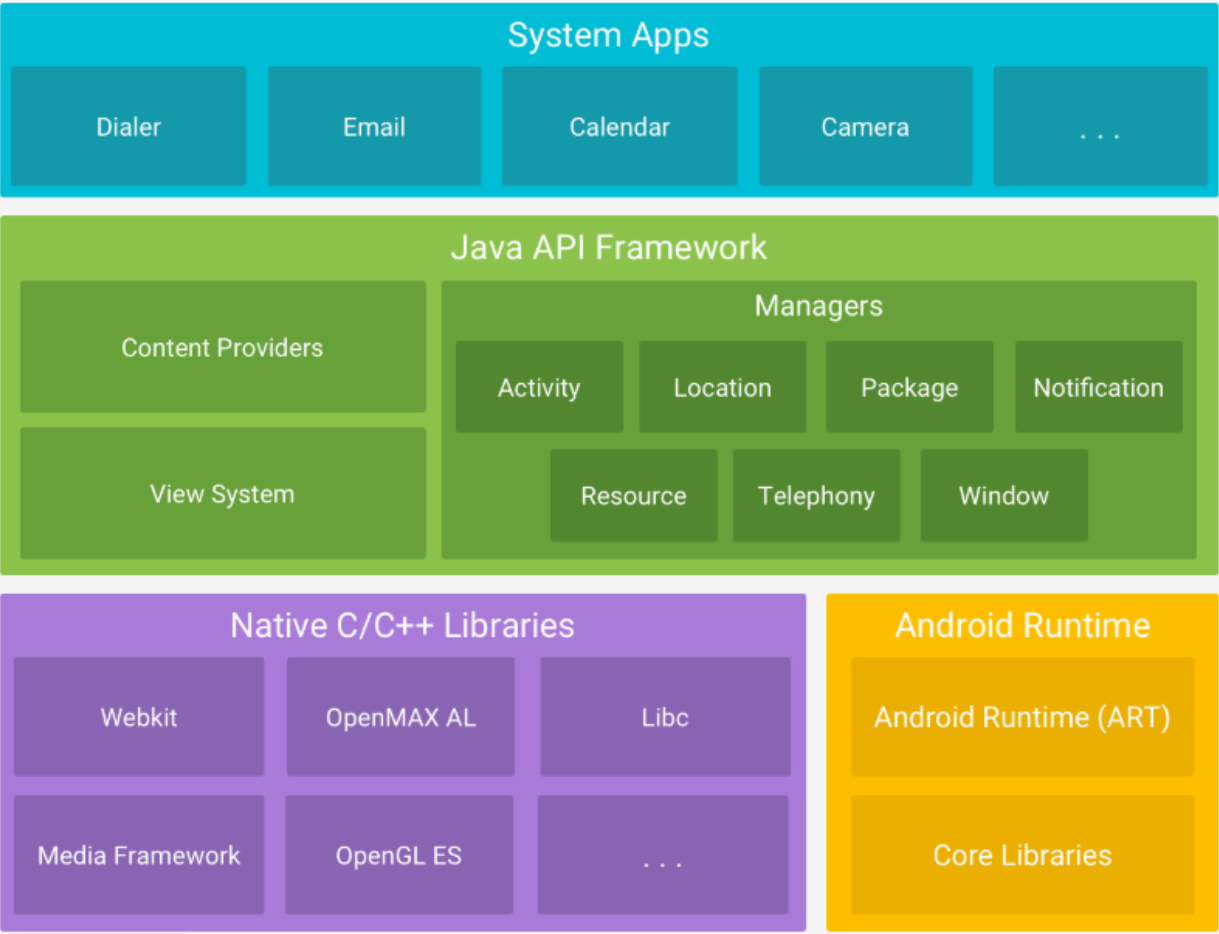
- The Android platform is currently being developed by Google
- The world's most popular operating system with 47% of the global market
- Based on Linux kernel
- Development can be done with Java and Kotlin programming languages (using Android SDK)
 - C, C++ (Android NDK)
- Cross-platform development
 - React Native (JS), Xamarin (C#), Flutter (Dart), etc.

Android versions and their shares

Name	Internal codename ^[9]	Version number(s)	API level	Linux kernel	Initial stable release date
Android 1.0	—	1.0	1	2.4.18	September 23, 2008
Android 1.1	Petit Four	1.1	2	2.4.22	February 9, 2009
Android Cupcake	Cupcake	1.5	3	2.6.27	April 27, 2009
Android Donut	Donut	1.6	4	2.6.29	September 15, 2009
Android Eclair	Eclair	2.0	5		October 27, 2009
		2.0.1	6		December 3, 2009
		2.1	7		January 11, 2010 ^[16]
Android Froyo	Froyo	2.2 – 2.2.3	8	2.6.32	May 20, 2010
Android Gingerbread	Gingerbread	2.3 – 2.3.2	9	2.6.35	December 6, 2010
		2.3.3 – 2.3.7	10		February 9, 2011
Android Honeycomb	Honeycomb	3.0	11	2.6.36	February 22, 2011
		3.1	12		May 10, 2011
		3.2 – 3.2.6	13		July 15, 2011
Android Ice Cream Sandwich	Ice Cream Sandwich	4.0 – 4.0.2	14	3.0.1	October 18, 2011
		4.0.3 – 4.0.4	15		December 16, 2011
Android Jelly Bean	Jelly Bean	4.1 – 4.1.2	16	3.0.31	July 9, 2012
		4.2 – 4.2.2	17	3.4	November 13, 2012
		4.3 – 4.3.1	18		July 24, 2013
Android KitKat	Key Lime Pie	4.4 – 4.4.4	19	3.10	October 31, 2013
		4.4W – 4.4W.2	20	3.14	June 25, 2014
Android Lollipop	Lemon Meringue Pie	5.0 – 5.0.2	21	3.16	November 4, 2014 ^[17]
		5.1 – 5.1.1	22	3.18	March 2, 2015 ^[18]
Android Marshmallow	Macadamia Nut Cookie	6.0 – 6.0.1	23	4.1	October 2, 2015 ^[19]
Android Nougat	New York Cheesecake	7.0	24	4.4	August 22, 2016
		7.1 – 7.1.2	25	4.9	October 4, 2016
Android Oreo	Oatmeal Cookie	8.0	26		August 21, 2017
		8.1	27	4.14	December 5, 2017
Android Pie	Pistachio Ice Cream ^[20]	9	28		August 6, 2018
Android 10	Quince Tart ^[21]	10	29	4.19	September 3, 2019
Android 11	Red Velvet Cake ^[21]	11	30	5.4	September 8, 2020
Android 12	Snow Cone	12	31	5.10	October 4, 2021
Android 12L	Snow Cone v2	12.1 ^[a]	32	5.15	March 7, 2022
Android 13	Tiramisu ^[23]	13	33		August 15, 2022

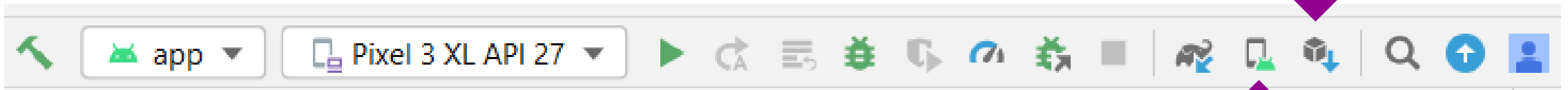
Legend: ■ Old version ■ Older version, still maintained ■ Latest version ■ Latest preview version





IDE

- ~~Formerly Eclipse + ADT plugin is used~~
- Currently **Android Studio** is the official IDE
 - <http://developer.android.com/sdk/index.html>
 - SDK Manager
 - AVD Manager
 - IntelliJ based IDE



SDK Manager

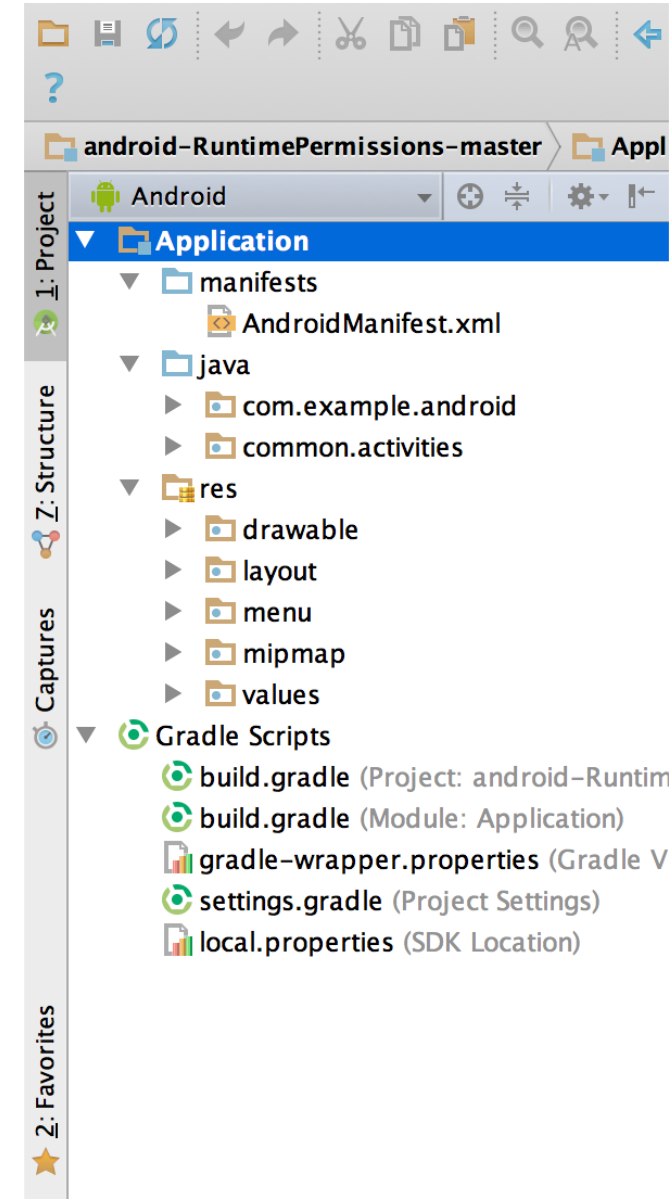
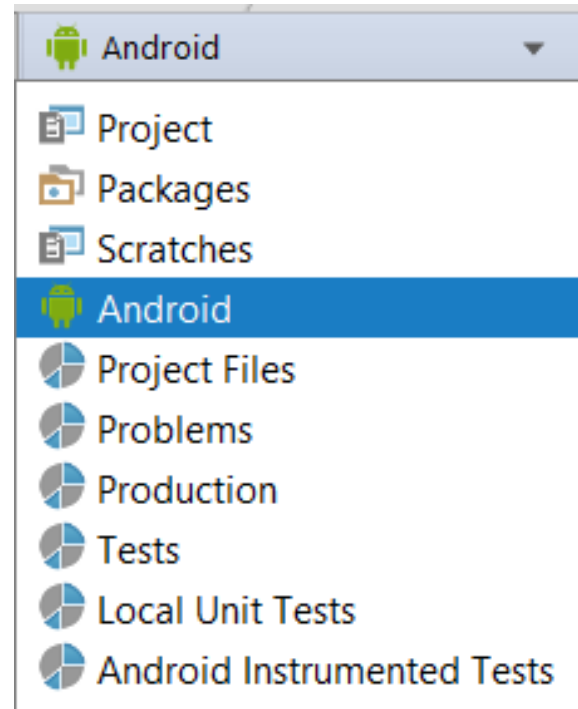
AVD Manager

Android Studio features

- A flexible Gradle-based build system
- A fast and feature-rich emulator
- A unified environment where you can develop for all Android devices
- Live Edit to update composables in emulators and physical devices in real time
- Code templates and GitHub integration to help you build common app features and import sample code
- Extensive testing tools and frameworks
- Lint tools to catch performance, usability, version compatibility, and other problems
- C++ and NDK support
- Built-in support for Google Cloud Platform, making it easy to integrate Google Cloud Messaging and App Engine

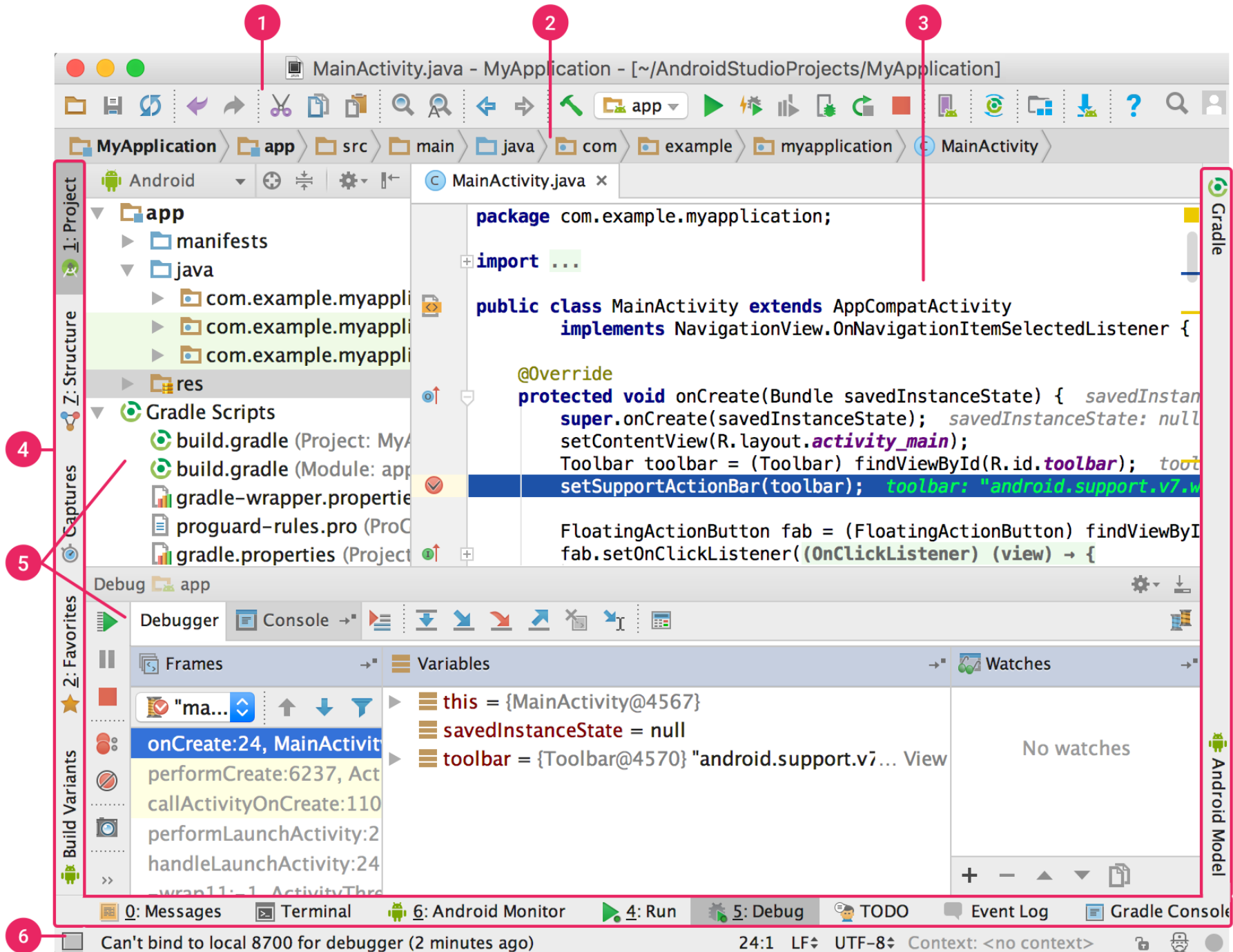
Android Studio – project structure

- Source code can be viewed in multiple views:
 - Android, Project, etc.



Android Studio – project structure

- Android view:
 - **Manifests**: consists of AndroidManifest.xml file
 - **java**: java source code
 - **res**: resource files: everything that are not java or other source code files
 - XML layouts
 - Text resources (strings.xml, dims.xml, arrays.xml, etc.)
 - Image, animation, audio, video



Android Studio – UI

- 1. Toolbar: run application, etc.
- 2. Navigation bar: navigate within a project; open files
- 3. Code editor: create and modify code files, design layouts
- 4. Tool window bar: other tools
- 5. Tool windows: special assets for version control, search, debugging, etc.
- 6. Status bar: displaying warnings, errors and other messages

Android Studio – UI

- Save current window and sidebar layout:
 - Window → Store Current Layout as Default
- Restore default window layout:
 - Window → Restore Default Layout
- Hide or show toolbars:
 - View → Enter Distraction Free Mode
 - View → Exit Distraction Free Mode

Android Studio – code-completion

- Quick help: Alt + Enter
 - E.g.: importing a package of an unknown type
- Android Studio provides three type of code-completion
 - Basic: completion of variables, types, methods and expressions
 - Smart: show relevant options based on existing code
 - Statement: completion of current expression, missing brackets, formatting, etc.

Android Studio – navigation

- Ctrl + B: Go to definition
- Ctrl + N: navigate to class (after entering the name of the class)
 - Make recommendations while typing
- Ctrl + Shift + N: navigate to file (directory, file search)
- Ctrl + Shift + Alt + N: navigate to symbol (ex.: variables)
- Alt + F7: searches for instances of a given element in the code
- Shift + Shift: search everywhere (among files)

Android Studio – formatting

- When editing the code, we also need to pay attention to formatting
- Android Studio provides automatic formatting:
 - Ctrl + Alt + L
 - Popup window displays how many lines have been modified
- Formatting style can be changed
 - File → Settings → Editor → Code Style

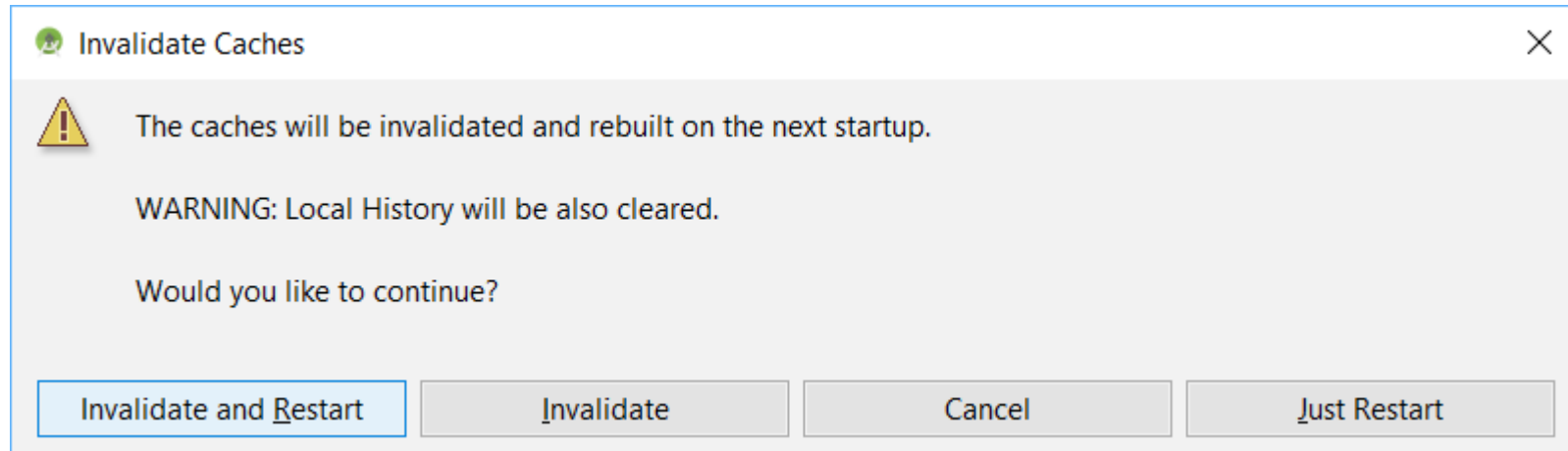
Android Studio – build, run

- Build
 - Make project
 - Clean project
 - Rebuild project
 - Build APK: creating apk file
 - Generate Signed APK: apk file for Google Play uploading



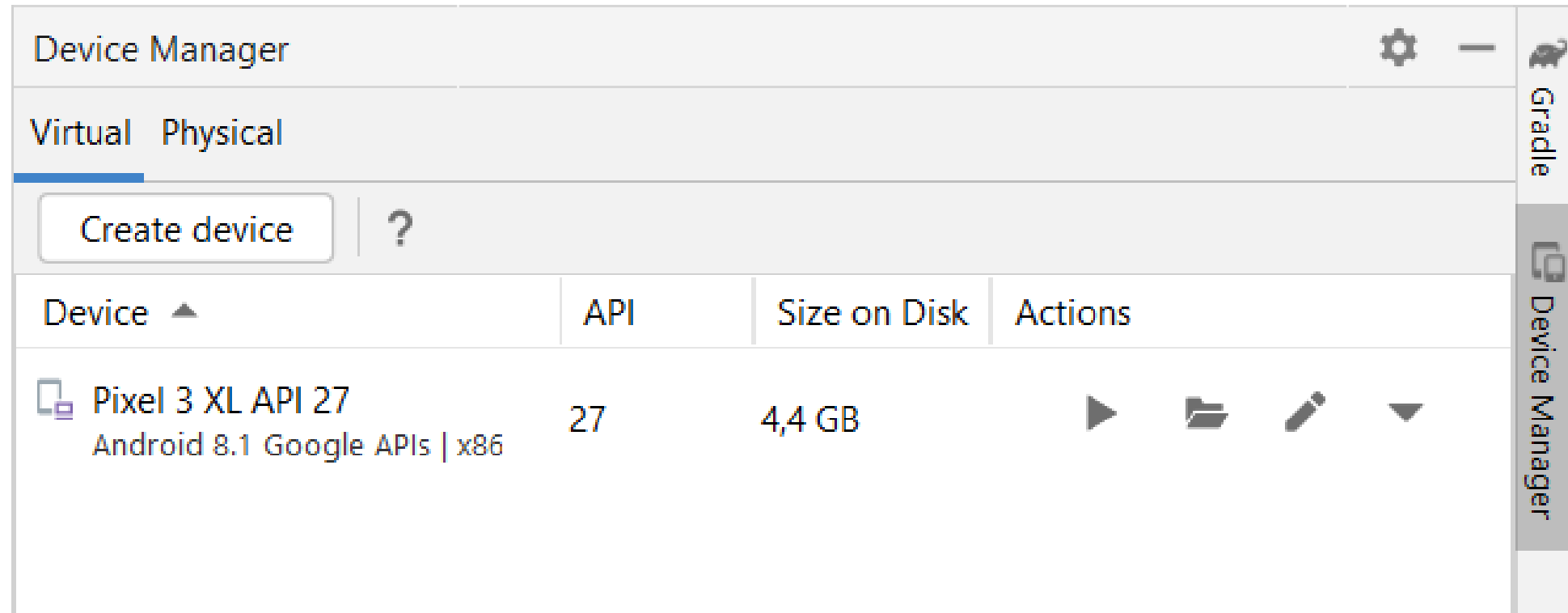
Android Studio – delete cache

- File → Invalidate Caches / Reset

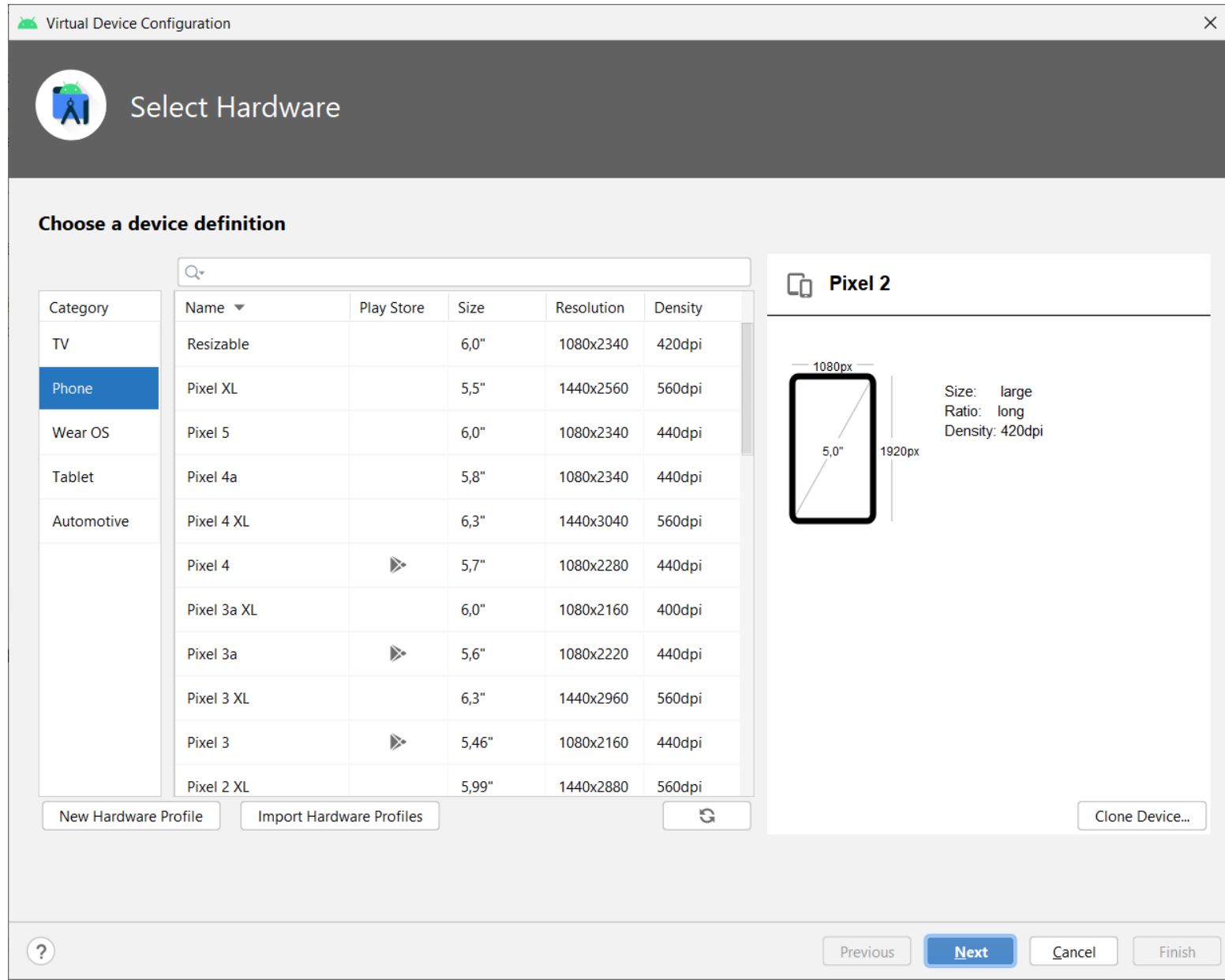


Android Studio - emulators

- AVD Manager



Android Studio - emulators



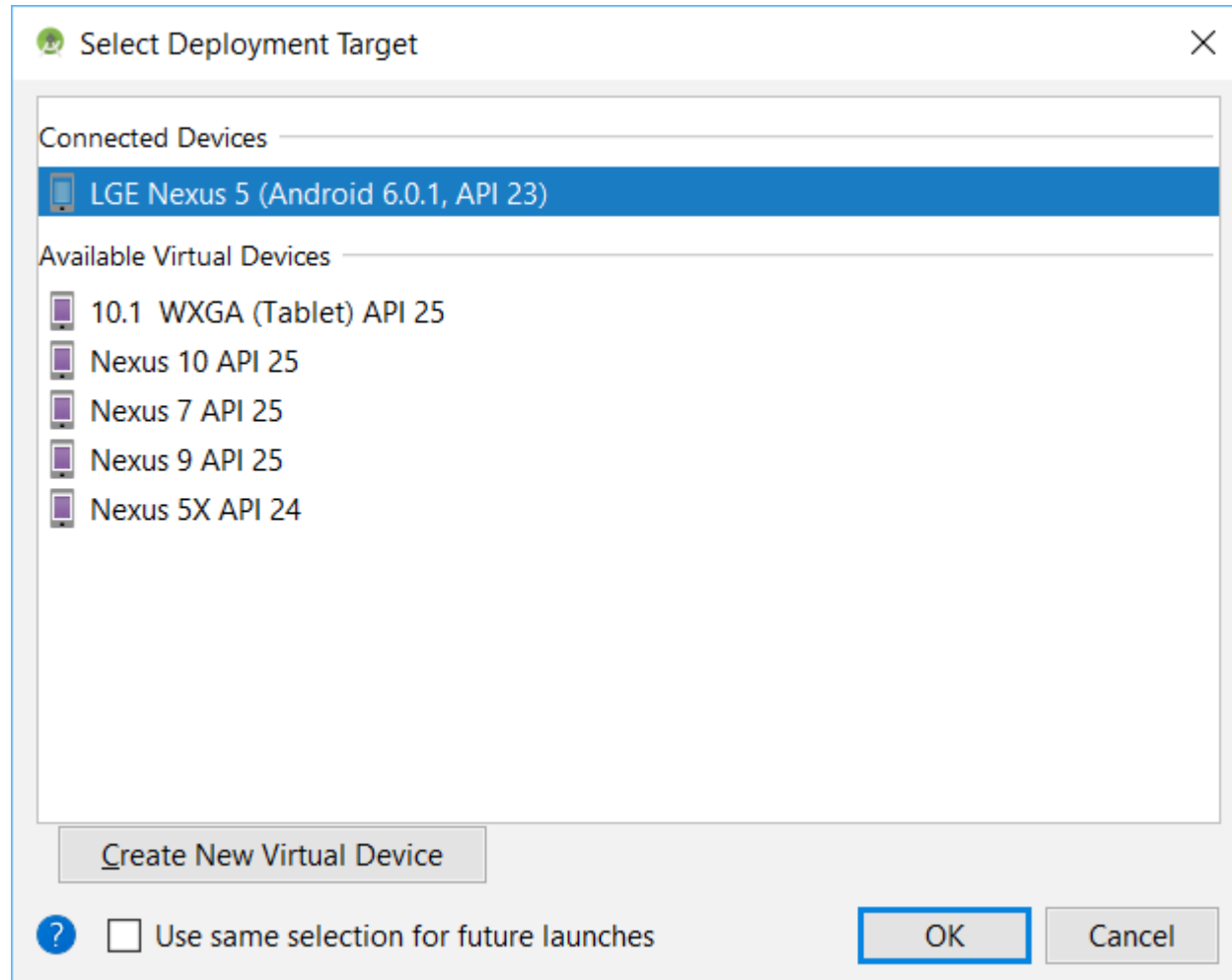
Running on physical device

1. Set up developer mode on the device
 1. Settings → About phone → Build number
 2. 7x tap on Build number, you will receive a toast message informing you that developer mode is now available
2. In the new developer menu we enable USB debugging
3. Some manufacturers also require the installation of their own software
 1. E.g.: LG
4. Connecting via USB to the development machine
5. In the device manager, check that the driver is properly installed

Running on physical device

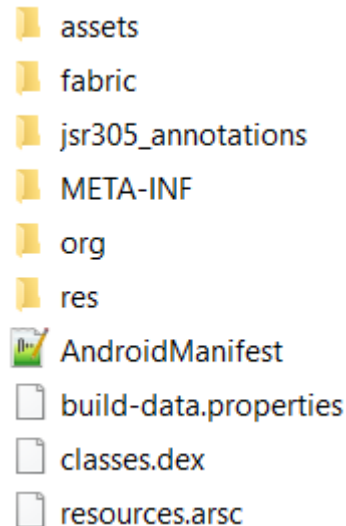
6. Android device asks if you want to enable the connected machine
7. In Android Studio, click on run and the connected device will appear in the window that opens
 1. OFFLINE requires additional settings on the device (shows options in Notification) or additional software to be installed on the developer machine (see previous slide LG example)

Running on physical device



Content of Apk file

- Once we've build our app, we get an apk file that can be installed on your Android device
 - Apps downloaded from Google Play will also come as an apk



Content of Apk file

- META-INF directory
- CERT.RSA: application certificate
- MANIFEST.MR: metainformation in key-value format
- CERT.SF: list of resources and their SHA-1 hash values
- Res directory: resources associated with the application
- AndroidManifest.xml: application name, version, permissions, libraries
- Classes.dex: compiled classes for ART
- Resources.arsc: resource data

Android application architecture

Android environment

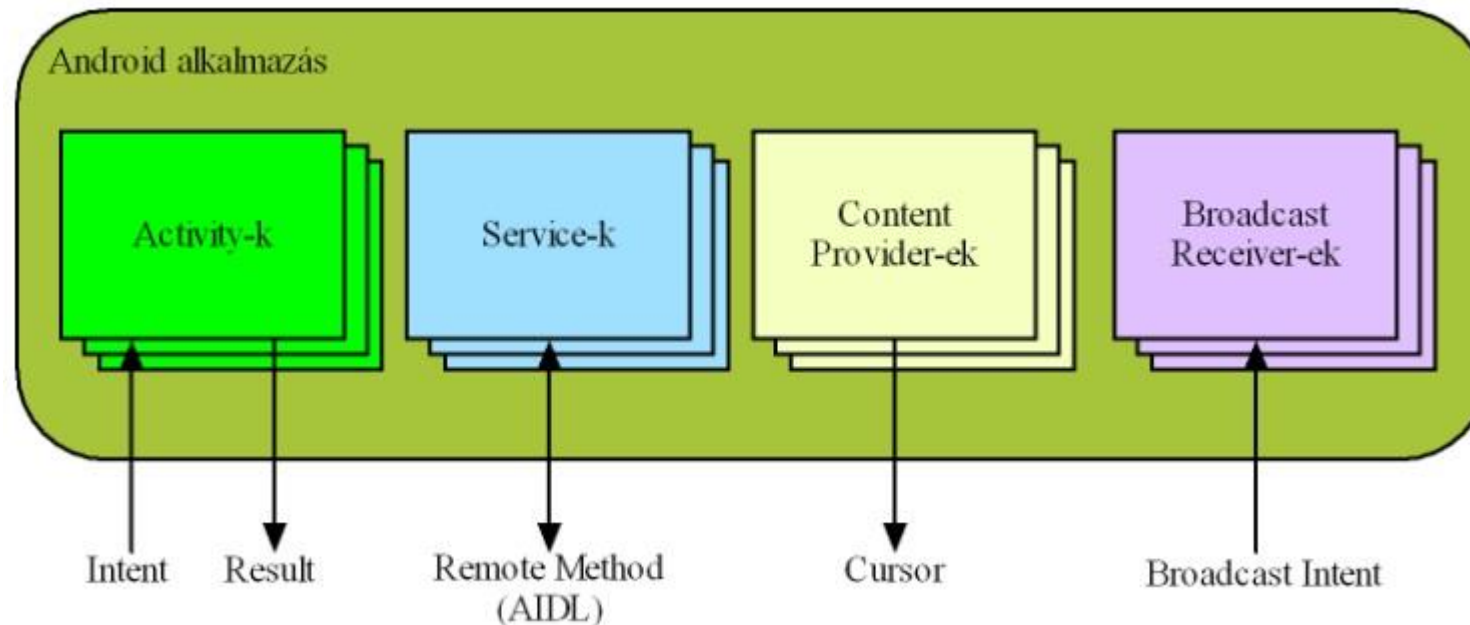
- All applications run in a sandbox after installation
- Each process has its own virtual machine instance, so the application code runs separately from the others
- Memory management is complex task on Android
 - Keeps applications on standby so that the user can access them as quickly as possible
 - When memory is running low, Android kill less important processes
- Minimum required privileges are given, everything else must be set in the application manifest file
 - This must be approved by the user at installation

App components

- An application can consist of several different types of components
- Each type serves a different purpose
- Four main components in Android:
 - **Activity**
 - **Service**
 - **ContentProvider**
 - **BroadcastReceiver**

App components

- One application may be activated by another application component without affecting the other components



Activity

- Most used application component
- Activity holds a user interface (layout)
 - There can be several Activities within one application
- Android gives you the option to have one application launch an Activity in another application
 - Flexible application design

Service

- Represents a task running in the background for a longer period of time
- No separate user interface, but you can launch Activities or show pop-up interface elements
- View running services from the system menu

Service

- Typical Service functions:
 - Synchronisation of content in the background
 - Position/usage tracking
 - Music playback
 - Downloading large content

Content Provider

- Content provider component, responsible for managing the data source and serving requests
- Data source can be anything (web, RSS, etc.)
- Its role is to hide the complexity and location of the data source
- Each Content Provider within the system has a unique URI that can be used to access it from other applications
- Example: Contacts, call log

Broadcast Receiver

- Its task is to be activated by different events and to perform a task
 - E.g.: low battery voltage, incoming call
- On a broadcast event, the system checks if there is a component involved and triggers them

Application architecture from a development perspective

- Four main groups:
 - Metainformation: mainly the application's project description manifest file
 - Resources: XML-based user interface, XML language files, multimedia elements, other XML
 - Source codes
 - External class libraries: compiled class libraries that are used by our application

Manifest file

- XML file, whose main task is to define the application components
- Here you can also specify the minimum requirements and privileges needed to run the application
- <http://developer.android.com/guide/topics/manifest/manifest-intro.html>

Manifest file

- Manifest consists of:
 - Package name (unique)
 - Version
 - Minimum API version (uses-sdk)
 - Permissions (uses-permission)
 - Use of external libraries (uses-library)
 - List of application components
Indication which application component handles which event (intent-filter)

Resource files

- Everything that is not a source file is considered a resource
- User interface and application logic are well separated
- res library called res contains the interface, images, sounds, etc.
- When using resources, the development environment automatically updates a file called R.java
 - Contains values of type Int, which assigns a unique value to each resource
 - Never modify!

References

- Meet Android Studio
 - <https://developer.android.com/studio/intro/>
- Projects overview
 - <https://developer.android.com/studio/projects/>